AESC / NeuroField v2: A Two-Speed Memory Architecture for Recurrent Language Models

Giridhari Karmakar
giriisdev@gmail.com

Technical research report · Project AERIS_64 · October 2026

Abstract

This report describes **NeuroField (AESC v2)**, a recurrent language-model architecture that combines a predictive dendrite, a k-step neural field, top-k skill routing, and a **two-speed memory**: fast weights written token-by-token by a surprise-gated delta rule, and slow weights consolidated by replay. The production code path uses a **parallel / looped form** (depth passes over the sequence with a causal fast-memory scan). We document forward equations, safety freeze behaviour, training practice, and known limitations. The architecture is operational in open-source code; scale and frontier-quality language performance remain open for future work.

1. Introduction

Large language models typically rely on stacked self-attention. AESC explores a complementary hypothesis: local predictive structure (dendrite), iterative state refinement (neural field), modular skills (router), and explicit fast/slow memory can support sequence modelling without a full T by T attention matrix. **AERIS_64** is the engineering product shell. This document is an honest snapshot: the design runs end-to-end (train, checkpoint, serve), but data scale and optimisation gaps still limit quality.

2. Architecture overview

```
tokens -> Embed
       -> Dendrite window (causal local past)
       -> for depth j = 1 .. k:
            Predict -> delayed error E
            Gate g (neuromodulator)
            Router / skills -> O
            Fast memory causal scan (delta rule)
            Neural field step
       -> Slow ring read (+ segment write)
       -> LayerNorm(O + scale * P_f(R) + P_s(U)) -> logits
```

3. Forward equations

3.1 Embedding

```
X_t = Embed(token_t)   in R^d
```

3.2 Dendrite (causal local window + delayed error)

Window width W, learned coefficients c_j, bias b, error gain gamma:

```
d_win_t = tanh( sum_{j=0}^{W-1} c_j * x_{t-j} + b )
d_t     = d_win_t + gamma * e_t
```

3.3 Prediction, error, and surprise

```
x_hat_{t+1} = f_p(h_t)
e_{t+1}     = stopgrad(x_{t+1}) - x_hat_{t+1}    # detached target
S_{t+1}     = stopgrad( mean |e_{t+1}| )
S_bar_t     = rho * S_bar_{t-1} + (1 - rho) * S_t
S_tilde_t   = S_t / (S_bar_t + eps)
```

3.4 Write gate (neuromodulator)

```
g_t = sigmoid( a_s * (S_tilde_t - 1) + a_c * mean(|h_t|) + b + delta_t )
g_t = min(g_t, g_max)

Safety freeze (Section 7 style):
  if memory_frozen:  g_t = 0
  => M only decays, no new writes.
```

3.5 Neural field (one step)

```
h <- (1 - zeta) * h
     + zeta * LayerNorm( alpha * h + beta * (d_t + R(tanh(h)) + W_m * r_{t-1}) )

R = shared local conv over neuron index
k_depth may be fixed or scheduled from relative surprise.
```

3.6 Router and skills

```
pi     = softmax(W_r * h)
w      = softmax(top-k logits of pi)
w_STE  = w + pi - stopgrad(pi)       # straight-through estimator
o_t    = sum_i  w_STE_i * Op_i(h)
```

3.7 Fast memory — decay-consistent delta rule (Change 5)

```
k_t = normalize(W_k * d_{t-1})
v_t = W_v * o_t
Lambda = diag(sigmoid(theta))          # elementwise decay in (0,1)
M_decay = Lambda * M_{t-1}
M_t = M_decay + g_t * k_t * (v_t - M_decay^T * k_t)^T
M_t <- M_t * min(1, M_max / ||M_t||_F)   # optional bound

Read:
q_t = normalize(W_q * d_t)
r_t = M_t^T * q_t

If g_t = 0:  M_t = Lambda * M_{t-1}   (decay only).
```

3.8 Slow memory and readout

```
u_t = SlowRead(d_t; ring slots)      # content-addressed ring
z_t = W_o * LayerNorm( o_t + s * P_f * r_t + P_s * u_t )
logits_t = z_t over vocabulary

Replay sketch:
W_s <- (1 - eta) * W_s + eta * sum_b (S^{(b)} / sum S) * M^{(b)}
```

3.9 Parallel / looped production path

The shipped forward pass embeds all T positions, builds dendrite windows in parallel, then runs k depth iterations. Within each depth, projections and routing are batched over T; the fast-memory delta rule remains a **causal left-to-right scan**. Each depth uses a fresh fast-memory matrix to avoid cross-depth leakage. Decode-time generation must match this math (parity tests).

```
X = Embed(tokens)                 # (B, T, d)
D_win = DendriteWindow(X)         # causal
H = D_win
for j in 1..k:
    E = stopgrad(X) - shift(Predict(H))
    D = D_win + gamma * E
    O = RouterSkills(H)           # all T
    M, R = DeltaScan_causal(D, O, G)
    H = FieldStep(H, D, shift(R))
logits = Readout(O, R, U_slow)
```

4. Causality

Unlike Transformers, AESC does not use a triangular attention mask. Causality is enforced by (i) the dendrite window over past embeddings only, (ii) one-step-shifted prediction error, (iii) ordered fast-memory writes, and (iv) shifted memory reads into the field. Changing x_{t+1} must not change logits at positions less than or equal to t (leakage tests).

5. Training

Training minimises next-token cross-entropy, plus optional error-energy and routing balance terms. Practical additions include label smoothing, dropout, warmup, and cosine or Noam-style learning-rate schedules. Data-parallel training via torchrun/DDP is supported. Throughput is typically lower than a pure Transformer of similar width because of the causal memory scan and depth loop.

```
L = CE(logits, targets; label_smoothing)
  + lambda_err * mean(||E||^2)
  + lambda_bal * routing_balance
```

6. Safety observability

Section-7 style design: anomaly detection can set a session flag memory_frozen, forcing g_t = 0 so that fast memory only decays. This avoids destructive permanent weight freezes while stopping further plastic writes under suspicious surprise patterns.

7. Implementation status

Open-source repository AERIS_64 provides: core modules (dendrite, field, memory, router, neuromodulator), parallel forward, checkpointing (safetensors), training entrypoints, FastAPI serve, optional tools, and tests (including leakage / decode parity where maintained). Small educational runs confirm loss decrease and causal checks; they are not claims of competitive language modelling.

8. Limitations and future work

The author acknowledges present gaps, to be addressed when time permits:

1. **Data scale and SFT quality** dominate perceived intelligence; architecture alone is not enough.
2. **Throughput:** causal delta scan + depth k makes large token budgets expensive.
3. **Gate collapse** (g near 0) can disable plasticity; needs monitoring and schedule tuning.
4. **Train vs decode parity** remains a recurring engineering risk under path changes.
5. **Skill utilisation** and long-context ring behaviour need multi-seed, harder tasks.
6. Chunkwise / fully parallel delta algorithms and stronger baselines vs Transformers are open.
7. Safety policy beyond freeze is thin; red-teaming incomplete.

9. Conclusion

NeuroField / AESC v2 is a coherent two-speed-memory recurrent design with explicit equations, a working codebase, and clear causality mechanisms. The architecture is suitable for continued research and small-scale experiments. Closing the gap to strong language performance requires data, compute, and the limitation list above—not a redesign of the core idea.

Author

Giridhari Karmakar
Email: giriisdev@gmail.com
Project: AERIS_64 / NeuroField (AESC)
Repository: https://github.com/giridhari-code/AERIS_64

This document is a technical research report for archival and engineering handoff. It does not claim peer-reviewed publication status.